Google Cloud H2S

AI Builder Cup

Build in JAPAC.
Build for the world.



## Team Details

a. Team name: **EmoldinoUnited**
b. Team leader name: **Utku Gulbardak (team member: Wonho Lee)**
c. Problem Statement: **Manufacturing, Intelligent Operations and Industrial Efficiency. A predicted failure only helps if the spare part arrives in time.**

## Brief about the idea

**At 2 a.m., two machines need the last spare part. OSI decides who gets it, covers the other, and proves it paid off.**

OSI (Opsight Intelligence) is an AI operations copilot for manufacturing. Predictive maintenance stops at the alert; OSI carries it across stores, purchasing and customer service.

A team of Gemini agents on Google's Agent Development Kit checks the spare part at both sites, the supplier's delivery record, and the customer orders at risk, prices every option, and drafts the work order, the parts action, and the customer note.

A person approves. The approval moves stock, and once the failure date passes, the outcome is scored against what the site would have done alone.

**$197,949**

net saved over four months, after charging the plan for false alerts

**662 h**

of downtime avoided on real failures

**8 / 8**

agent evaluation scenarios passed with Gemini on Vertex AI

# Opportunities

HOW IS IT DIFFERENT?

## It works between departments

Predictive maintenance stops at the alert, and most AI copilots answer one department's question.

OSI's agents mirror a plant's morning meeting: maintenance, procurement and customer service work in parallel, then an operations lead decides.

HOW DOES IT SOLVE THE PROBLEM?

## It answers the three questions

Is the spare on the shelf or promised elsewhere? When will the open order really arrive? Which customer orders are at risk?

Every option is priced as extra spend plus lost contribution, and the plan is drafted for approval.

USP

## A plan by morning, and proof

The autopilot triages each alert as it arrives, so the plan is ready at 2 a.m.

Approvals change the world, false alerts are charged against the plan, and every number an agent quotes is checked against the tools.

# List of features offered by the solution

### Live mode and autopilot

The plant replays in time. Each new alert pauses the clock while the agents triage it, so a plan waits for approval.

### Control room

Alerts, spare conflicts, every option priced, sensor evidence against the normal band, and customers at risk.

### Agent team copilot

Three specialists in parallel, then a lead. Every tool call and brief streams live with the agent's name.

### Supplier inbox

Gemini reads supplier emails and PDFs into schema-checked delivery updates; a delay moves the open order everywhere.

### Photo inspection

Gemini vision checks a technician's photo against the predicted wear before money is spent on a part.

### Approvals and track record

A person approves; stock moves. Every alert is scored at its real outcome, and operators can mark false alerts.

Orange cards use Gemini.

# Process flow diagram or Use-case diagram

## 1 Sense

A gradient-boosting model scores every machine from its sensors.

## 2 Reason

Gemini agents check sensors, photos, spares, the supplier, and customer orders.

## 3 Act

The team drafts the work order, parts action, and customer note. A person approves.

## 4 Close the loop

Approval moves stock and clears conflicts; the outcome is scored later.

**ONE ALERT, START TO FINISH · 25 JULY 2026**

### 25 Jul 02:00

Two spindle-bearing alerts at US1 need the one bearing set on the shelf.

### Clock paused

Supplier on time for 66.1% of lines; Gemini reads its delay notice: 8 August.

### Plan

Air transfer from site DE1 for $650. Waiting: 190 h down, 4 customer lines at risk.

### 26 Jul 10:00

Approved. The spindle fails as predicted, and the part is already there.

**26 h** down instead of 230 h **$100,077** saved on this one alert

# Wireframes/Mock diagrams of the proposed solution (optional)

**SEVEN PAGES, ONE DEMO CLOCK**

| Page | What it shows |
|---|---|
| Live | The plant replayed in time, with the autopilot |
| Control room | Alerts, spare conflicts, priced options, sensor evidence, customers at risk |
| Copilot | Questions to one agent or to the whole team |
| Supplier inbox | Supplier messages read into delivery updates |
| Approvals | Approve or reject; outcomes appear later |
| Track record | Every alert scored at its real outcome |
| How it works | The loop, the architecture, the guardrails |

### Watch the team work

Each tool call and brief streams live, not a spinner.

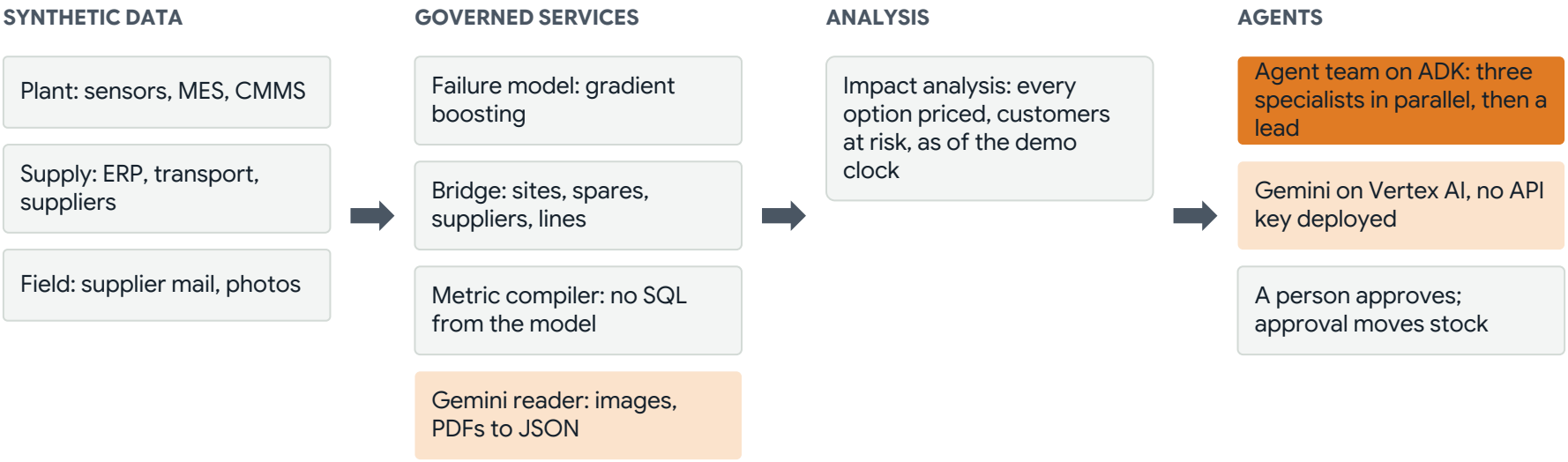
### A verified badge

Shown only when every number matches the tools.

### Errors in plain words

Quota, overload, or credentials, each with what to do.

# Architecture diagram of the proposed solution




**Google Cloud** One Cloud Run container serves FastAPI, the live agent stream, and the React app. Gemini runs on Vertex AI under a service account that may only call Vertex AI. Deployed from source with Cloud Build.

Orange uses Gemini. Prediction, pricing and metrics are ordinary code, so every number an agent quotes is computed, then checked.

# Technologies to be used in the solution

## Google Cloud and AI

**Gemini 3.8 Flash**
Agent reasoning, multimodal reading of PDFs and photos; backup models on quota

**Vertex AI**
Gemini endpoint under the service's own account; no API key deployed

**Agent Development Kit**
ParallelAgent of three specialists inside a SequentialAgent with the lead

**Cloud Run**
One container, scales to zero; one warm instance during judging

**Cloud Build, Artifact Registry**
Deploy from source with one script

## Application stack

**FastAPI, server-sent events**
API and the live agent stream

**React, TypeScript**
The seven-page app

**scikit-learn**
Gradient-boosting failure model

**DuckDB, Parquet, pandas**
Plant and supply data, generated from seeds

**Governed YAML layer**
Metric definitions and the site bridge

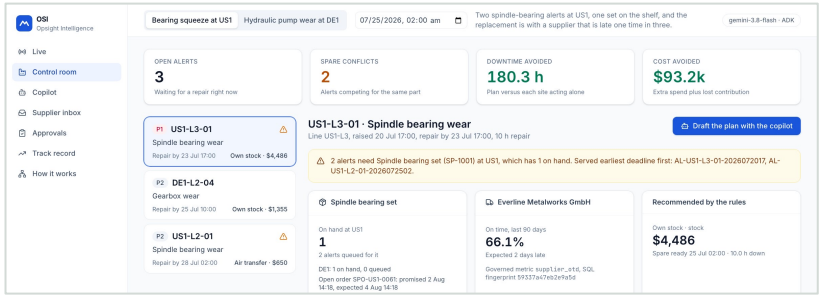
# Estimated implementation cost (optional)

**THE PROTOTYPE RUNS ON THE GOOGLE CLOUD FREE TRIAL**

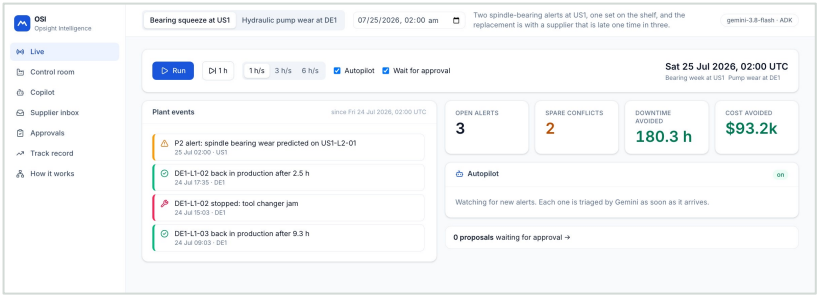
| Item | Prototype | How it is bounded |
|---|---|---|
| Cloud Run | One container, one instance at most | Scales to zero when idle; one warm instance only during judging |
| Gemini Flash | Cents per agent run | A team run is about 12 model calls, a single run about 5 |
| Usage caps | 10 runs per visitor, 60 per hour in total | Nobody can drain the credit; the autopilot stops itself |
| Runaway agents | 25 model calls per run, 50 per team run | A confused agent cannot loop on the credit |
| Build and registry | Cloud Build, Artifact Registry | Pay per build; images deleted on teardown |

At plant scale the agents run per alert, a few a day per site, not per sensor reading, so the Gemini cost grows with alerts. The data services added at scale (Firestore, BigQuery, Pub/Sub) are pay-per-use.

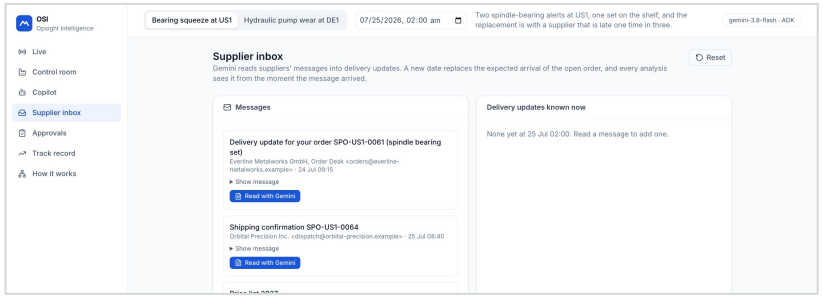
# Snapshots of the prototype



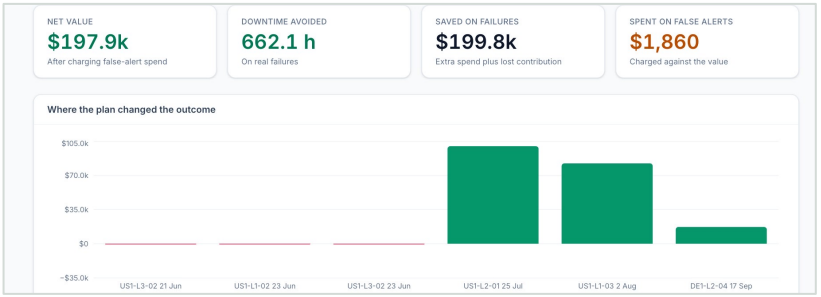

Control room: the alert, the spare conflict, every option priced




Live: the plant replays and the autopilot triages each alert




Supplier inbox: Gemini reads messages and PDFs




Track record: every alert scored at its real outcome

# Prototype Performance report/Benchmarking

**27 of 41**

failures caught; median warning 86 h; 0.16 false alerts a machine-month

**$197,949**

net saved over four months; 662 h of downtime avoided

**8 / 8**

agent evaluation scenarios on Gemini 3.8 Flash, Vertex AI

**1-2 min**

for the full agent team to plan an alert on the deployed app

## EVERY ALERT OF FOUR MONTHS, SCORED AGAINST WHAT REALLY HAPPENED

| Alerts scored | Plans that differed | Saved on real failures | Spent on false alerts | Net |
|---|---|---|---|---|
| 38 (26 real, 12 false) | 6 of 38 | $199,809 | $1,860 | $197,949 |

The evaluation scores the tools called, the required facts, the number check, and the proposal. 446 backend and 15 frontend tests, and offline tests of the real ADK runner.

Reproduce: make models (failure model), make value (track record), make eval (agents).

# Additional Details/Future Development (if any)

| Part | Prototype today | At scale |
|---|---|---|
| Operational state | JSON files in the container | Firestore |
| Plant and supply data | DuckDB over Parquet, from seeds | BigQuery, fed from MES, CMMS, ERP |
| Sensor and event stream | Replayed from history | Pub/Sub |
| Failure model | Trained at build time | Vertex AI training and endpoints |
| Monitoring | Container logs | Cloud Monitoring for agent latency |
| More sites | Two sites, four supply plants | One mapping file per site |

## Honest limits

Synthetic data: a two-site plant and a four-plant supply network, generated from seeds.

Spare costs and contribution per unit are declared assumptions.

Approvals reset when the container restarts, until Firestore.

Connecting a real plant means mapping its data into the same governed tables; the agents, tools, guardrails and evaluation carry over unchanged.

## Provide links to your:

- **GitHub Public Repository**: https://github.com/UttyWotty/ai_build_base_2026
- **Demo Video Link (3 Minutes)**: https://youtu.be/b1bc3dhXC74
- **Final Product Link**: https://ops-copilot-432875717420.asia-southeast1.run.app

OSI (Opsight Intelligence) · EmoldinoUnited: Utku Gulbardak (lead), Wonho Lee

The video runs 2:10. Every number in this deck can be reproduced from the public repository.

Google Cloud H2S

# AI Builder Cup

Build in JAPAC.
Build for the world.

# Thank you

